

W3SCHOOLS REACT TUTORIAL · CSAE 104 WEB PROGRAMMING

Complete React Development Guide

Based on: <https://www.w3schools.com/react/>

Topics Covered:

ES6 Essentials · JSX · Components · Props · Events · Conditionals
Lists · Forms · CSS Styling · React Router · Higher-Order Components
Hooks: useState · useEffect · useContext · useRef · useReducer · useCallback · useMemo
Custom Hooks · Portals · Suspense · Sass · Transitions · Forward Ref

Don Mariano Marcos Memorial State University – South La Union Campus
College of Computer Science | CSAE 104 Web Programming

Prepared by: Prof. Haydee D. Limson

UNIT 1 GETTING STARTED — React Intro, Setup & First App

1.1 What is React?

React is an open-source JavaScript library built by Facebook for creating fast, interactive user interfaces. It was first used internally in 2011 for Facebook's News Feed, then open-sourced in July 2013. React 19.0 was released in December 2024.

Feature	What It Means
Component-based UI	Build small, reusable UI pieces — like LEGO bricks for your app
Virtual DOM	React only re-renders what changed — making apps very fast
One-way data flow	Data flows from parent to child — predictable and easy to debug
React Native	Same React skills extend to iOS and Android mobile apps
Huge ecosystem	Largest front-end community; thousands of compatible libraries

⚠ Note Prerequisites

You must know HTML, CSS, and intermediate JavaScript (functions, arrays, objects, promises). If unsure, review [W3Schools JavaScript](#) before continuing.

1.2 Getting Started — Two Ways

Option A React Directly in HTML (CDN — no install)

The quickest way to try React without installing anything. Add three script tags to any HTML file. Good for experiments only — not for real projects.

INDEX.HTML — REACT VIA CDN (NO INSTALL NEEDED)

```
<!DOCTYPE html>
<html lang="en">
<head><title>My React App</title></head>
<body>

  <!-- React mounts into this div -->
  <div id="root"></div>

  <!-- Step 1: React library -->
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <!-- Step 2: ReactDOM to render to browser -->
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
  <!-- Step 3: Babel to compile JSX in the browser -->
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

  <script type="text/babel">
    function Hello() {
```

```

    return <h1>Hello, React!</h1>;
  }
  const root = ReactDOM.createRoot(document.getElementById("root"));
  root.render(<Hello />);
</script>
</body>
</html>

```

Option B Create React App (CRA) — Professional Setup

The standard way to start a real React project. Automatically sets up Webpack, Babel, a dev server, and hot reloading.

TERMINAL — CREATE AND START YOUR REACT APP

```

# Make sure Node.js is installed first
node --version      # must be v16 or higher
npm --version       # comes bundled with Node.js

# Create a new React project called "my-app"
npx create-react-app my-app

# Move into the project folder
cd my-app

# Start the development server
npm start

# Browser opens automatically at http://localhost:3000

```

Project Folder Structure

GENERATED FOLDER STRUCTURE EXPLAINED

```

my-app/
├── node_modules/      ← installed packages (never edit)
├── public/
│   └── index.html      ← only HTML file; has <div id="root">
├── src/
│   ├── index.js        ← entry point – renders App into HTML
│   ├── App.js          ← your root component (start here)
│   ├── App.css         ← styles for App.js
│   └── index.css       ← global styles
├── package.json       ← project info and dependency list
└── .gitignore

```

SRC/INDEX.JS — THE REACT ENTRY POINT

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

```

```
// createRoot() is the React 18 way to mount your app
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

SRC/APP.JS — YOUR MAIN COMPONENT (REPLACE WITH YOUR OWN CODE)

```
function App() {
  return (
    <div>
      <h1>Hello, TheFolio!</h1>
      <p>My first React app.</p>
    </div>
  );
}

export default App;
```

✓ Tip Hot Reload

Every time you save a file, the browser automatically refreshes — you never need to manually reload. This is called Hot Module Replacement (HMR).

🔗 Exercise 1: Setup & Hello World

1. Install Node.js from nodejs.org — verify with: `node --version`
2. Run: `npx create-react-app student-blog` then `cd student-blog` then `npm start`
3. Open `src/App.js` and change the content to display your name and course
4. Add a second paragraph with today's date — save and watch the browser update instantly
5. Run: `npm run build` — see the optimized production files in the `/build` folder

UNIT 2 ES6 ESSENTIALS — Modern JavaScript React Requires

React code is written with modern JavaScript (ES6+). These features appear constantly in React — master them before writing components.

2.1 Classes

ES6 classes provide cleaner syntax for objects with methods. React class components are built on them, but modern React uses functional components + hooks.

ES6 CLASS — SYNTAX AND INHERITANCE

```
class Animal {
  constructor(name, sound) {
    this.name = name; // property
    this.sound = sound;
  }
  speak() {
    return `${this.name} says ${this.sound}!`;
  }
}

// Inheritance with extends
class Dog extends Animal {
  constructor(name) {
    super(name, "woof"); // call parent constructor
  }
  fetch() { return `${this.name} fetches!`; }
}

const d = new Dog("Rex");
console.log(d.speak()); // Rex says woof!
console.log(d.fetch()); // Rex fetches!
```

2.2 Arrow Functions

Shorter function syntax used everywhere in React — event handlers, array methods, useEffect callbacks.

ARROW FUNCTION — ALL FORMS

```
// Regular function
function add(a, b) { return a + b; }

// Arrow function - same thing, shorter
const add = (a, b) => a + b;

// Single param - parentheses optional
const double = n => n * 2;
```

```
// No params - parentheses required
const greet = () => "Hello!";

// Multi-line - needs { } and return
const multiply = (a, b) => {
  const result = a * b;
  return result;
};
```

✓ Tip Arrow & this

Arrow functions do NOT have their own "this". They inherit it from the enclosing scope. This is why they work perfectly inside React class components and event handlers.

2.3 Variables — let and const

Keyword	When to Use
var	Function-scoped, hoisted. AVOID — never use in React
let	Block-scoped, can be reassigned. Use for values that change (counters, loop vars)
const	Block-scoped, cannot be reassigned. Use for EVERYTHING else in React

LET AND CONST EXAMPLES

```
const name = "Maria";      // cannot reassign the variable itself
let count = 0;              // can reassign
count++;                    // OK

// const with objects - variable stays fixed, properties can change
const user = { name: "Maria", age: 20 };
user.age = 21;              // OK - modifying a property
// user = {};               // ERROR - cannot reassign const

// const with arrays
const items = ["a", "b"];
items.push("c");            // OK - modifying array contents
```

2.4 Array map()

.map() creates a new array by transforming every element. It is the standard way to render lists in React JSX.

ARRAY MAP() — DATA TRANSFORMATION AND JSX RENDERING

```
const nums = [1, 2, 3, 4, 5];

// Transform every element
const doubled = nums.map(n => n * 2); // [2, 4, 6, 8, 10]

// Extract a field from objects
```

```
const students = [
  { id:1, name:"Ana",  score:88 },
  { id:2, name:"Ben",  score:75 },
  { id:3, name:"Cara", score:92 },
];
const names = students.map(s => s.name); // ["Ana","Ben","Cara"]

// In React - render a list of JSX elements
function StudentList() {
  return (
    <ul>
      {students.map(s => (
        <li key={s.id}>{s.name} - Score: {s.score}</li>
      ))}
    </ul>
  );
}
```

⚠ Note key prop

Always add `key={unique_id}` to the root element inside `.map()`. Without it, React cannot efficiently update the list and you will see a warning.

2.5 Destructuring

Extract values from arrays or objects into variables in a single line. Used constantly with props, `useState`, and `useEffect`.

OBJECT DESTRUCTURING

```
const person = { name:"Maria", age:20, city:"Manila" };

// Without destructuring
const name = person.name;
const age  = person.age;

// With destructuring - same result, one line
const { name, age } = person;

// Rename while destructuring
const { name: fullName } = person;

// Default value (used if property is undefined)
const { country = "Philippines" } = person;

// Destructuring in function parameters (React props pattern)
function Card({ title, description, color = "blue" }) {
  return <div style={{ color }}><h2>{title}</h2><p>{description}</p></div>;
}
```

ARRAY DESTRUCTURING — HOW USESTATE USES IT

```
const colors = ["red", "green", "blue"];
const [first, second] = colors; // first="red", second="green"
```

```
// Skip elements with commas
const [,third] = colors; // third="blue"

// Swap values without a temp variable
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1

// useState uses array destructuring - this is why it looks like this:
const [count, setCount] = useState(0);
//      ^ current value   ^ updater function
```

2.6 Spread Operator

The spread operator (...) expands arrays or objects. Used constantly in React state updates to copy and modify without mutating the original.

SPREAD WITH ARRAYS AND OBJECTS

```
// Combine arrays
const a = [1, 2];
const b = [3, 4];
const c = [...a, ...b];           // [1, 2, 3, 4]

// Copy and add an element
const newItem = [...items, "new item"];

// Spread objects - copy + override specific properties
const user = { name:"Maria", age:20, role:"user" };
const updated = { ...user, age:21 };
// { name:"Maria", age:21, role:"user" }

// React state update pattern - NEVER mutate state directly
const [form, setForm] = useState({ name:"", email:"" });

// Update just one field, keep the rest unchanged
setForm(prev => ({ ...prev, email:"new@email.com" }));
```

2.7 Modules — import & export

Every React component file uses ES6 modules to share code between files.

BUTTON.JS — NAMED AND DEFAULT EXPORTS

```
// Named export - can export multiple things per file
export const COLORS = { primary:"#1B3A6B", danger:"#C0392B" };

export function formatDate(date) {
  return new Date(date).toLocaleDateString();
}

// Default export - ONE per file, used for components
```

```
function Button({ text, onClick, variant="primary" }) {
  return (
    <button
      onClick={onClick}
      style={{ background: COLORS[variant] }}
    >
      {text}
    </button>
  );
}

export default Button;
```

APP.JS — IMPORTING

```
import Button from './Button'; // default import
import { COLORS, formatDate } from './Button'; // named imports
import { formatDate as fmtDate } from './Button'; // rename on import
import * as utils from './utils'; // import all as object
```

2.8 Ternary Operator & Template Strings

TERNARY — SHORTHAND IF/ELSE USED IN JSX

```
// condition ? valueIfTrue : valueIfFalse
const status = score >= 75 ? "Passed" : "Failed";

// In JSX — render different elements
function Status({ score }) {
  return (
    <span>
      {score >= 75 ? <b style={{color:"green"}}>Passed</b>
        : <b style={{color:"red"}}>Failed</b>}
    </span>
  );
}

// && — render OR render nothing
{isAdmin && <button>Delete All</button>}
```

TEMPLATE STRINGS — EMBED EXPRESSIONS IN STRINGS

```
const name = "Maria";
const score = 95;

// Old way — concatenation (messy)
const msg1 = "Hello, " + name + "! Score: " + score;

// Template string — clean and readable
const msg2 = `Hello, ${name}! Score: ${score}`;

// Any expression works inside ${}
const msg3 = `Grade: ${score >= 90 ? "A" : "B"}`;
```

```
// Multi-line strings
const html = `
  <div>
    <h1>${name}</h1>
  </div>
`;
```

🔗 Exercise 2: ES6 Essentials Practice

1. Write an arrow function `isPassing(score)` that returns `true` if `score >= 75`
2. Destructure `{ name, subject, score }` from an object and build a template string sentence
3. Use spread to add a new student to an array of students without modifying the original
4. Use `.map()` on `[10,20,30,40,50]` to produce `["10%","20%","30%","40%","50%"]`
5. Create a `utils.js` file that exports two named functions and import them in `App.js`

UNIT 3 JSX — JAVASCRIPT XML — Writing HTML Inside JavaScript

3.1 JSX Introduction

JSX (JavaScript XML) lets you write HTML-like syntax directly in JavaScript. Babel compiles it into `React.createElement()` calls — you never call `createElement` manually.

WHAT JSX COMPILES TO BEHIND THE SCENES

```
// What YOU write:
const element = <h1 className="title">Hello!</h1>;

// What BABEL compiles it into:
const element = React.createElement("h1", { className:"title" }, "Hello!");

// JSX can represent complex trees:
const card = (
  <div className="card">
    <h2>Title</h2>
    <p>Body text</p>
    <button>Click me</button>
  </div>
);
```

3.2 JSX Rules

Rule	Explanation
One root element	Wrap multiple elements in <code><div></code> or a Fragment: <code><> </></code>
Close all tags	<code></code> , <code>
</code> , <code><input /></code> — every tag must close
className not class	Use <code>className="..."</code> — <code>class</code> is a reserved JS word
htmlFor not for	In <code><label htmlFor="id"></code> — <code>for</code> is a reserved JS word
camelCase events	<code>onClick</code> , <code>onChange</code> , <code>onSubmit</code> — NOT <code>onclick</code> , <code>onchange</code>
{ } for expressions	Embed JS values with <code>{}</code> — <code>{name}</code> , <code>{count + 1}</code> , <code>{fn()}</code>
// Comments in JSX	Use <code>{/* comment */}</code> inside JSX, not <code>//</code> or <code><!-- --></code>

JSX RULES — ALL IN ONE COMPONENT

```
function Demo() {
  const user = "Maria";
  const score = 92;

  return (
    <>                                     {/* Fragment — no extra div */}
      {/* This is a JSX comment */}
      <h1>Hello, {user}!</h1>             {/* {} embeds JS */}
    </>
  );
}
```

```

    <p>Score: {score}/100 = {score}%</p>
    <p>{score} >= 90 ? "Excellent!" : "Good job!"</p>
        {/* self-closing */}
    <input
      className="form-input"                {/* className not class */}
      style={{ border:"1px solid gray", padding:"8px" }}
      type="text"
    />
    <label htmlFor="email">Email</label>    {/* htmlFor not for */}
    <input id="email" type="email" />
  </>
);
}

```

3.3 JSX Expressions

Curly braces {} let you embed any JavaScript expression inside JSX — variables, math, method calls, ternaries, and array maps.

JSX EXPRESSIONS — WHAT CAN GO INSIDE {}

```

function StudentCard({ student }) {
  return (
    <div>
      {/* Variables */}
      <h2>{student.name}</h2>

      {/* Math */}
      <p>Average: {(student.score / 100 * 100).toFixed(1)}%</p>

      {/* String methods */}
      <p>ID: {student.id.toString().padStart(4, "0")}</p>

      {/* Ternary */}
      <span>{student.score} >= 75 ? "✓ Passed" : "✗ Failed"</span>

      {/* && short-circuit */}
      {student.score} >= 90 && <span className="badge">Honor</span>

      {/* Array .map() */}
      <ul>
        {student.subjects.map((sub, i) => <li key={i}>{sub}</li>)}
      </ul>
    </div>
  );
}

```

3.4 JSX Attributes

JSX ATTRIBUTES — HTML VS JSX DIFFERENCES

```
// 1. className (not class)
<div className="container active">...</div>

// 2. Inline styles - double braces: outer{} = JSX, inner{} = JS object
//   Property names are camelCase: fontSize not font-size
<p style={{ color:"red", fontSize:"18px", marginTop:"10px" }}>Alert</p>

// 3. Dynamic values in attributes
const imgUrl = "/images/profile.jpg";
<img src={imgUrl} alt="Profile" width={200} height={200} />

// 4. Boolean attributes
<input type="checkbox" checked={true} disabled={false} />

// 5. Event handlers - camelCase
<button onClick={() => alert("clicked!")}>Click Me</button>
<input onChange={(e) => console.log(e.target.value)} />

// 6. htmlFor on labels
<label htmlFor="username">Username:</label>
<input id="username" type="text" />
```

3.5 Conditional Rendering in JSX

THREE PATTERNS FOR CONDITIONAL RENDERING

```
// Pattern 1: Ternary (most common - for either/or)
function AuthButton({ isLoggedIn }) {
  return isLoggedIn
    ? <button>Log Out</button>
    : <button>Log In</button>;
}

// Pattern 2: && (render or nothing)
function Dashboard({ notifications }) {
  return (
    <div>
      <h1>Dashboard</h1>
      {notifications > 0 && <p>You have {notifications} alerts</p>}
    </div>
  );
}

// Pattern 3: if/else before return (cleanest for complex logic)
function Greeting({ hour }) {
  let msg;
  if (hour < 12)      msg = "Good morning!";
  else if (hour < 18) msg = "Good afternoon!";
  else               msg = "Good evening!";
  return <h1>{msg}</h1>;
}
```

Exercise 3: JSX Practice

1. Create a JSX card component showing your name, course, and a profile picture placeholder
2. Apply className and inline styles (background, padding, border-radius, box-shadow)
3. Add a conditional: show a gold star ★ if score ≥ 90 , silver star if score ≥ 75 , nothing if below
4. Render a list of 5 subjects using `.map()` with a key prop
5. Create a toggle component that shows "Show Details" or "Hide Details" using ternary based on a boolean variable

UNIT 4 COMPONENTS & PROPS — Building Reusable UI

4.1 Functional Components

A component is a JavaScript function that returns JSX. Components let you split the UI into independent, reusable pieces — like custom HTML tags.

CREATING AND USING COMPONENTS

```
// components/Card.js — a simple reusable component
function Card({ title, body, tag }) {
  return (
    <div className="card">
      <span className="tag">{tag}</span>
      <h3>{title}</h3>
      <p>{body}</p>
    </div>
  );
}
export default Card;

// App.js — using the component multiple times
import Card from './components/Card';

function App() {
  return (
    <div className="grid">
      <Card title="React" body="A UI library" tag="Tech" />
      <Card title="MongoDB" body="A NoSQL database" tag="DB" />
      <Card title="Node.js" body="Server JavaScript" tag="Backend" />
    </div>
  );
}
```

✓ Tip Naming rule

Component names MUST start with an uppercase letter. <Card /> = your component. <card /> = an unknown HTML tag.

4.2 Class Components

The original React component type. Modern code uses functional components, but you will see class components in older codebases.

CLASS COMPONENT — EQUIVALENT TO THE FUNCTIONAL VERSION

```
import React, { Component } from 'react';

class Welcome extends Component {
  // State defined in constructor
  constructor(props) {
    super(props); // always required
    this.state = { count: 0 };
  }
}
```

```

    }

    // Arrow function methods - avoids "this" binding issues
    increment = () => {
      this.setState(prev => ({ count: prev.count + 1 }));
    };

    // render() is required - returns JSX
    render() {
      return (
        <div>
          <h1>Hello, {this.props.name}!</h1>
          <p>Clicks: {this.state.count}</p>
          <button onClick={this.increment}>Click</button>
        </div>
      );
    }
  }
}
export default Welcome;

```

4.3 Props

Props (properties) are how you pass data into a component from its parent — like function arguments. Props are read-only.

PROPS — PASSING DATA TO COMPONENTS

```

// Define the component with props
function StudentCard({ name, score, subject, isPassing }) {
  return (
    <div style={{ border: "1px solid #ccc", padding: "16px", margin: "8px" }}>
      <h3>{name}</h3>
      <p>{subject}: {score}/100</p>
      <p>Status: {isPassing ? "Passed ✓" : "Failed ✗"}</p>
    </div>
  );
}

// Use the component with different data
function App() {
  return (
    <>
      <StudentCard name="Ana" score={88} subject="Math" isPassing={true} />
      <StudentCard name="Ben" score={60} subject="Science" isPassing={false} />
      <StudentCard name="Cara" score={95} subject="English" isPassing={true} />
    </>
  );
}

```

4.4 Props Destructuring

Destructure props directly in the function signature for cleaner, shorter code.

THREE WAYS TO ACCESS PROPS — ALL EQUIVALENT

```
// Way 1: props object (verbose)
function Card(props) {
  return <h2>{props.title}</h2>;
}

// Way 2: destructure in the body (common)
function Card(props) {
  const { title, description, color = "blue" } = props;
  return <h2 style={{ color }}>{title}</h2>;
}

// Way 3: destructure in the parameter (RECOMMENDED)
function Card({ title, description, color = "blue" }) {
  return (
    <div style={{ color }}>
      <h2>{title}</h2>
      <p>{description}</p>
    </div>
  );
}
```

4.5 Props Children

The special children prop lets you pass JSX content between a component's opening and closing tags — like innerHTML.

CHILDREN PROP — WRAPPING CONTENT IN COMPONENTS

```
// A Panel component that wraps any content
function Panel({ title, children }) {
  return (
    <div className="panel">
      <div className="panel-header"><h3>{title}</h3></div>
      <div className="panel-body">
        {children}    {/* anything placed between <Panel>...</Panel> */}
      </div>
    </div>
  );
}

// Usage — children is everything between the tags
function App() {
  return (
    <Panel title="Student Info">
      <p>Name: Maria Santos</p>
      <p>Course: BSCS</p>
      <button>Edit Profile</button>
    </Panel>
  );
}
```

```
}
```

Exercise 4: Components & Props

1. Create a PostCard component that accepts: title, excerpt, author, date, and tag as props
2. Add a default value for tag: "General" using destructuring default syntax
3. Create a Badge component using the children prop — wrap text between `<Badge>Passed</Badge>`
4. In App.js, render 4 PostCards with different data by mapping over an array of objects
5. Style PostCard with a border, rounded corners, and a color-coded tag using inline styles

UNIT 5 EVENTS, LISTS & CONDITIONALS — Interactivity and Dynamic Rendering

5.1 React Events

React event handlers work like HTML events but use camelCase naming and receive a SyntheticEvent object. Pass a function reference — do NOT call the function inline.

EVENT HANDLING — COMMON PATTERNS

```
function EventDemo() {
  // Named handler function — clean and reusable
  const handleClick = () => alert("Button clicked!");

  const handleChange = (e) => {
    console.log("Input value:", e.target.value);
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // prevent page reload
    console.log("Form submitted!");
  };

  return (
    <form onSubmit={handleSubmit}>
      {/* Pass the function reference — NOT handleClick() */}
      <button onClick={handleClick}>Click Me</button>

      <input onChange={handleChange} placeholder="Type here" />

      {/* Inline arrow — OK for simple one-liners */}
      <button onClick={() => alert("Inline!")}>Inline</button>

      <button type="submit">Submit</button>
    </form>
  );
}
```

Event	When It Fires
onClick	Button, div, any element click
onChange	Input, select, textarea value changes
onSubmit	Form submission
onFocus / onBlur	Input gains or loses focus
onMouseEnter / onMouseLeave	Hover effects
onKeyDown / onKeyUp	Keyboard key pressed or released

5.2 React Lists

Use `.map()` to render arrays as lists of JSX elements. Always include a unique key prop on each rendered element.

RENDERING A LIST WITH `.MAP()`

```
const products = [
  { id:1, name:"Laptop", price:45000, inStock:true },
  { id:2, name:"Mouse", price:800, inStock:true },
  { id:3, name:"Monitor", price:12000, inStock:false },
];

function ProductList() {
  return (
    <ul>
      {products.map(product => (
        // key must be unique among siblings
        <li key={product.id} style={{ color: product.inStock ? "black" :
"gray" }}>
          {product.name} - ₪{product.price.toLocaleString()}
          {!product.inStock && <span> (Out of Stock)</span>}
        </li>
      ))}
    </ul>
  );
}
```

⚠ Note key prop rules

Keys must be unique among siblings (not globally). Use database IDs, not array indices, as keys whenever possible. The index as key (`key={i}`) can cause bugs when items are reordered.

5.3 React Conditionals

Control what gets rendered based on state or props using these three patterns:

ALL THREE CONDITIONAL PATTERNS COMPARED

```
// PATTERN 1: Ternary — either A or B
function ToggleButton({ isOn }) {
  return <button>{isOn ? "Turn Off" : "Turn On"}</button>;
}

// PATTERN 2: && — show or show nothing
function Alert({ message, hasError }) {
  return (
    <div>
      {hasError && <p className="error">{message}</p>}
    </div>
  );
}

// PATTERN 3: if before return — best for complex logic
```

```
function UserGreeting({ user }) {  
  if (!user) return <p>Loading...</p>;  
  if (user.role === "admin") return <h1>Admin Dashboard</h1>;  
  return <h1>Welcome, {user.name}!</h1>;  
}
```

Exercise 5: Events, Lists & Conditionals

1. Create a button that logs "Hello!" to the console when clicked
2. Create an input that shows its current value in a <p> below it as you type (use onChange)
3. Render a list of 5 students from an array showing name and score — highlight rows where score < 75 in red
4. Add a "Remove" button next to each student that filters them out of the array
5. Use conditional rendering to show "No students" when the array is empty

UNIT 6 REACT FORMS — Controlled Inputs & Form Handling

6.1 Controlled Inputs

In React, form inputs are "controlled" — their value is stored in state and updated via `onChange`. React is the single source of truth for the input's value.

CONTROLLED INPUT — BASIC PATTERN

```
import { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!email || !password) {
      setError("All fields are required.");
      return;
    }
    console.log("Submitting:", { email, password });
    setError("");
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Login</h2>
      {error && <p style={{ color:"red" }}>{error}</p>}

      <label>Email:</label>
      <input
        type="email"
        value={email} // controlled: tied to state
        onChange={e => setEmail(e.target.value)}
      />

      <label>Password:</label>
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
      />

      <button type="submit">Login</button>
    </form>
  );
}
```

6.2 Multiple Inputs — One Handler

Use the input name attribute and a single onChange to handle many fields with one state object.

MULTIPLE INPUTS WITH SHARED STATE OBJECT

```
function RegisterForm() {
  const [form, setForm] = useState({
    name: "", email: "", password: "", course: "",
  });

  // One handler for all inputs - uses e.target.name
  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Form data:", form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange}
placeholder="Full Name" />
      <input name="email" value={form.email} onChange={handleChange}
placeholder="Email" type="email" />
      <input name="password" value={form.password} onChange={handleChange}
placeholder="Password" type="password" />
      <input name="course" value={form.course} onChange={handleChange}
placeholder="Course" />
      <button type="submit">Register</button>
    </form>
  );
}
```

6.3 Textarea, Select, Checkbox, Radio

SPECIAL FORM CONTROLS — REACT SYNTAX

```
// Textarea - value prop instead of content between tags
<textarea
  value={bio}
  onChange={e => setBio(e.target.value)}
  rows={4}
/>

// Select dropdown
<select value={course} onChange={e => setCourse(e.target.value)}>
  <option value="">-- Select Course --</option>
  <option value="bscs">BSCS</option>
  <option value="bsit">BSIT</option>
  <option value="bsis">BSIS</option>
</select>
```

```

</select>

// Checkbox - uses checked not value
<input
  type="checkbox"
  checked={isAgree}
  onChange={e => setIsAgree(e.target.checked)}
/>

// Radio buttons - same name, different values
<input type="radio" name="gender" value="male"
  checked={gender === "male"}
  onChange={e => setGender(e.target.value)} />
<input type="radio" name="gender" value="female"
  checked={gender === "female"}
  onChange={e => setGender(e.target.value)} />

```

6.4 React Portals

Portals let you render a component outside its parent DOM node — typically used for modals, tooltips, and overlays that need to escape parent overflow/z-index.

REACTDOM.CREATEPORTAL() — MODAL EXAMPLE

```

import { createPortal } from 'react-dom';

function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  // Renders into document.body, not inside the parent component
  return createPortal(
    <div style={{
      position:"fixed", inset:0, background:"rgba(0,0,0,0.5)",
      display:"flex", alignItems:"center", justifyContent:"center",
    }}>
      <div style={{ background:"white", padding:"2rem", borderRadius:"8px" }}>
        {children}
        <button onClick={onClose}>Close</button>
      </div>
    </div>,
    document.body // mount target - outside the React root
  );
}

// Usage
function App() {
  const [open, setOpen] = useState(false);
  return (
    <>
      <button onClick={() => setOpen(true)}>Open Modal</button>
      <Modal isOpen={open} onClose={() => setOpen(false)}>
        <h2>Modal Title</h2>
        <p>Modal content here</p>
      </Modal>
    </>
  );
}

```

```
    </Modal>
  </>
);
}
```

Exercise 6: Forms Practice

1. Build a registration form: name, email, password, confirm password, course (select), agree to terms (checkbox)
2. Add validation: all fields required, password length ≥ 8 , passwords must match
3. Show error messages in red below each invalid field
4. On successful submit, show a success message and clear the form
5. Build a modal that opens when you click "Preview Registration" showing the submitted data

UNIT 7 CSS STYLING & REACT ROUTER — Styling Approaches and Navigation

7.1 CSS Styling Approaches

Method 1 Plain CSS File — className

APP.CSS + CLASSNAME — SIMPLEST APPROACH

```
/* App.css */
.card {
  background: white;
  border: 1px solid #e0e0e0;
  border-radius: 8px;
  padding: 1.5rem;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
}
.card__title { font-size: 1.25rem; font-weight: 700; }
.card__tag    { color: #c0392b; font-size: 0.75rem; text-transform: uppercase; }

/* App.js — import and use classNames */
import './App.css';
function Card({ title, tag }) {
  return (
    <div className="card">
      <span className="card__tag">{tag}</span>
      <h3 className="card__title">{title}</h3>
    </div>
  );
}
```

Method 2 CSS Modules — scoped styles

CSS Modules automatically scope class names to the component — no naming conflicts between files.

BUTTON.MODULE.CSS + BUTTON.JS

```
/* Button.module.css */
.btn          { padding: 10px 20px; border-radius: 6px; cursor: pointer; }
.primary      { background: #1B3A6B; color: white; }
.danger       { background: #c0392b; color: white; }

/* Button.js */
import styles from './Button.module.css';

function Button({ text, variant="primary" }) {
  return (
    <button className={`${styles.btn} ${styles[variant]}`}>
      {text}
    </button>
  );
}

// Generated class names: "Button_btn__x1Yz Button_primary__Ab3c"
```

Method 3 Inline Styles — JavaScript objects

INLINE STYLES — DYNAMIC VALUES

```
function Alert({ type, message }) {
  const styles = {
    padding: "12px 16px",
    borderRadius: "6px",
    marginBottom: "1rem",
    // Dynamic color based on type prop
    background: type === "error" ? "#ffebee" : "#e8f5e9",
    color:      type === "error" ? "#c0392b" : "#1e8449",
    border:     `1px solid ${type === "error" ? "#ffcdd2" : "#c8e6c9"}`,
  };
  return <div style={styles}>{message}</div>;
}
```

Method 4 Sass — CSS with superpowers

Sass adds variables, nesting, and mixins to CSS. Install with: `npm install sass`

STYLES.SCSS — SASS SYNTAX IN REACT

```
// Install: npm install sass
// File extension: .scss

$primary: #1B3A6B;
$accent:  #C0392B;
$radius:  8px;

.card {
  border: 1px solid #e0e0e0;
  border-radius: $radius;
  padding: 1.5rem;

  // Nesting
  &__title { color: $primary; font-weight: 700; }
  &__tag   { color: $accent; font-size: 0.8rem; }

  &:hover { box-shadow: 0 4px 20px rgba(0,0,0,0.12); }
}

// In your component:
import './styles.scss';
```

7.2 React Router

React Router (v6) enables navigation between pages in a Single Page Application (SPA) without full page reloads.

TERMINAL — INSTALL REACT ROUTER

```
npm install react-router-dom
```

APP.JS — SETTING UP ROUTES

```

import { BrowserRouter, Routes, Route, Link, NavLink, Navigate } from 'react-router-dom';
import HomePage      from './pages/HomePage';
import AboutPage     from './pages/AboutPage';
import PostPage      from './pages/PostPage';
import NotFoundPage  from './pages/NotFoundPage';

function App() {
  return (
    <BrowserRouter>
      {/* Navigation */}
      <nav>
        <NavLink to="/"      className={({ isActive }) => isActive ? "active" : ""}>Home</NavLink>
        <NavLink to="/about">About</NavLink>
      </nav>

      {/* Route definitions */}
      <Routes>
        <Route path="/"      element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
        {/* Dynamic route - :id is a URL parameter */}
        <Route path="/post/:id" element={<PostPage />} />
        {/* Redirect */}
        <Route path="/home"  element={<Navigate to="/" />} />
        {/* Catch all - 404 */}
        <Route path="*"      element={<NotFoundPage />} />
      </Routes>
    </BrowserRouter>
  );
}

```

POSTPAGE.JS — READING URL PARAMETERS

```

import { useParams, useNavigate } from 'react-router-dom';

function PostPage() {
  const { id } = useParams(); // reads :id from URL
  const navigate = useNavigate(); // programmatic navigation

  return (
    <div>
      <h1>Post #{id}</h1>
      <button onClick={() => navigate("/")}>Back to Home</button>
      <button onClick={() => navigate(-1)}>Go Back</button>
    </div>
  );
}

```

Component / Hook	Purpose
<BrowserRouter>	Wraps the entire app — provides routing context
<Routes>	Container for all <Route> elements

<code><Route path="/x" element={<C />}></code>	Maps a URL path to a component
<code><Link to="/x"></code>	Navigation link — no page reload
<code><NavLink to="/x"></code>	Link with active state styling
<code><Navigate to="/x"></code>	Programmatic redirect in JSX
<code>useParams()</code>	Hook — read :param values from URL
<code>useNavigate()</code>	Hook — navigate programmatically in code

Exercise 7: CSS Styling & Router

1. Convert your blog app to use CSS Modules — create `Card.module.css` and import it
2. Add React Router: Home, About, Contact, and a dynamic `post/:id` route
3. Style the `NavLink` active state (bold + colored underline when on that page)
4. Create a `NotFound` (404) page that shows when no route matches
5. Pass a post ID via `useNavigate` from the home page to the post detail page

UNIT 8 REACT HOOKS — PART 1 — [useState](#) · [useEffect](#) · [useContext](#)

8.1 What are Hooks?

Hooks are functions that let functional components use state and lifecycle features previously only available in class components. All hooks start with "use". They must be called at the top level of a component — never inside loops, conditions, or nested functions.

Hook	Purpose
useState	Store and update component state
useEffect	Run side effects (fetch, timers, subscriptions)
useContext	Access global state without prop drilling
useRef	Access DOM elements or persist values without re-render
useReducer	Complex state logic (alternative to useState)
useCallback	Memoize a function reference
useMemo	Memoize an expensive computed value

8.2 useState

useState is the most fundamental hook. It adds reactive state to a functional component — when state changes, React re-renders the component.

USESTATE — COMPLETE PATTERNS

```
import { useState } from 'react';

function Counter() {
  // useState returns [currentValue, setterFunction]
  const [count, setCount] = useState(0);    // number
  const [name, setName] = useState("");    // string
  const [isOpen, setIsOpen] = useState(false); // boolean
  const [items, setItems] = useState([]);   // array
  const [user, setUser] = useState(null);   // object/null

  // Simple update
  const increment = () => setCount(count + 1);

  // Functional update — use when new state depends on previous
  const safeIncrement = () => setCount(prev => prev + 1);

  // Update array state — spread to avoid mutation
  const addItem = (item) => setItems(prev => [...prev, item]);
  const removeItem = (id) => setItems(prev => prev.filter(i => i.id !== id));

  // Update object state — spread to keep other fields
```

```

const updateName = (newName) =>
  setUser(prev => ({ ...prev, name: newName }));

return (
  <div>
    <p>Count: {count}</p>
    <button onClick={increment}>+1</button>
    <button onClick={() => setIsOpen(o => !o)}>Toggle</button>
    {isOpen && <p>Panel is open</p>}
  </div>
);
}

```

⚠ Note Never mutate state directly

setCount(count + 1) is correct. count++ is WRONG — React will not detect the change and will not re-render.

8.3 useEffect

useEffect runs code after every render (or specific renders). Use it for data fetching, setting up subscriptions, updating the document title, and other side effects.

USEEFFECT — FOUR DEPENDENCY PATTERNS

```

import { useState, useEffect } from 'react';

function PostList() {
  const [posts, setPosts] = useState([]);
  const [count, setCount] = useState(0);

  // 1. No dependency array - runs after EVERY render
  useEffect(() => {
    document.title = `Count: ${count}`;
  });

  // 2. Empty array [] - runs ONCE after first render (on mount)
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts?_limit=5")
      .then(res => res.json())
      .then(data => setPosts(data));
  }, []); // <-- empty array

  // 3. With dependency [count] - runs when count changes
  useEffect(() => {
    console.log("Count changed to:", count);
  }, [count]);

  // 4. Cleanup - runs before next effect or component unmount
  useEffect(() => {
    const timer = setInterval(() => setCount(c => c+1), 1000);
    return () => clearInterval(timer); // cleanup function
  }, []);

  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}

```

}

Pattern	When It Runs
<code>useEffect(() => {}, [])</code>	Runs once — on mount. Good for initial data fetch.
<code>useEffect(() => {})</code>	Runs after every render. Rarely needed.
<code>useEffect(() => {}, [x])</code>	Runs when x changes.
<code>return () => cleanup()</code>	Cleanup: cancel timers, remove listeners on unmount.

8.4 useContext

`useContext` provides a way to share data across the component tree without passing props through every level — called "prop drilling". Perfect for user auth, themes, and language settings.

USECONTEXT — CREATING AND CONSUMING CONTEXT

```
// Step 1: Create the context (usually a separate file)
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext("light"); // default value

// Step 2: Provide the context to all child components
function App() {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
      <MainContent />
    </ThemeContext.Provider>
  );
}

// Step 3: Consume in any child - no prop drilling needed!
function Header() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <header style={{ background: theme === "dark" ? "#333" : "#fff" }}>
      <button onClick={() => setTheme(t => t === "dark" ? "light" : "dark")}>
        Toggle {theme === "dark" ? "Light" : "Dark"} Mode
      </button>
    </header>
  );
}

// Works at ANY depth - no need to pass theme as prop
function DeepComponent() {
  const { theme } = useContext(ThemeContext);
  return <div className={`card ${theme}`}>Content</div>;
}
```

 Exercise 8: useState, useEffect, useContext

1. Build a counter with +, -, and Reset buttons using useState
2. Add a useEffect that updates the browser tab title to "Count: [n]" whenever count changes
3. Build a PostFetcher that fetches from <https://jsonplaceholder.typicode.com/posts> on mount and displays 10 post titles
4. Create a ThemeContext with "light" and "dark" modes — apply background and text color throughout the app
5. Build a live search that filters a list of students as you type using useState and useEffect

UNIT 9 REACT HOOKS — PART 2 — useRef · useReducer · useCallback · useMemo

9.1 useRef

useRef returns a mutable ref object with a .current property. It does NOT cause re-renders when changed. Use it to access DOM elements directly, or to persist values between renders.

USEREF — TWO COMMON USES

```
import { useRef, useEffect } from 'react';

// USE 1: Access a DOM element directly
function AutoFocusInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // focus the input on mount
  }, []);

  const handleClear = () => {
    inputRef.current.value = "";
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} placeholder="Auto-focused!" />
      <button onClick={handleClear}>Clear</button>
    </div>
  );
}

// USE 2: Persist a value across renders WITHOUT causing re-renders
function TimerComponent() {
  const [count, setCount] = useState(0);
  const intervalRef = useRef(null); // store timer ID

  const start = () => {
    intervalRef.current = setInterval(() => setCount(c => c+1), 1000);
  };

  const stop = () => clearInterval(intervalRef.current);

  return (
    <div>
      <p>Seconds: {count}</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}
```

9.2 useReducer

useReducer is an alternative to useState for complex state logic — multiple related values, or next state depends on current in complex ways. It is the same idea as Redux.

USEREDUCER — SHOPPING CART EXAMPLE

```
import { useReducer } from 'react';

// The reducer function: takes state + action, returns new state
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return { ...state, items: [...state.items, action.payload],
        total: state.total + action.payload.price };

    case 'REMOVE_ITEM':
      const removed = state.items.find(i => i.id === action.payload);
      return { ...state,
        items: state.items.filter(i => i.id !== action.payload),
        total: state.total - (removed?.price || 0) };

    case 'CLEAR':
      return { items: [], total: 0 };

    default: return state;
  }
}

const initialState = { items: [], total: 0 };

function ShoppingCart() {
  const [cart, dispatch] = useReducer(cartReducer, initialState);

  const addLaptop = () => dispatch({
    type: 'ADD_ITEM',
    payload: { id: 1, name: "Laptop", price: 45000 },
  });

  return (
    <div>
      <button onClick={addLaptop}>Add Laptop</button>
      <button onClick={() => dispatch({ type: "CLEAR" })}>Clear</button>
      <p>Total: ₱{cart.total.toLocaleString()}</p>
      <ul>
        {cart.items.map(item => (
          <li key={item.id}>
            {item.name}
            <button onClick={() => dispatch({ type: 'REMOVE_ITEM',
payload:item.id })}>
              Remove
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```

    ))}
  </ul>
</div>
);
}

```

9.3 useCallback

`useCallback` memoizes a function so it is not recreated on every render. Use it when passing callbacks to optimized child components or when a function is a `useEffect` dependency.

USECALLBACK — PREVENTING UNNECESSARY RE-RENDERS

```

import { useState, useCallback, memo } from 'react';

// memo() - child only re-renders if props change
const Button = memo(function Button({ onClick, children }) {
  console.log("Button rendered");
  return <button onClick={onClick}>{children}</button>;
});

function Parent() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  // Without useCallback: new function on every render → Button re-renders
  // With useCallback: same function reference → Button does NOT re-render
  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []); // no dependencies = created once

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <p>Count: {count}</p>
      <Button onClick={handleClick}>Increment</Button>
    </div>
  );
}

```

9.4 useMemo

`useMemo` memoizes the **RESULT** of an expensive computation. It only recalculates when its dependencies change.

USEMEMO — EXPENSIVE CALCULATIONS

```

import { useState, useMemo } from 'react';

const students = [ /* array of 1000 students */ ];

```

```

function StudentReport() {
  const [filter, setFilter] = useState("all");
  const [search, setSearch] = useState("");

  // Without useMemo: recalculates on every render (even unrelated state
  // changes)
  // With useMemo: recalculates only when filter or search changes
  const filteredStudents = useMemo(() => {
    console.log("Filtering students..."); // see when this runs
    return students
      .filter(s => filter === "all" || s.status === filter)
      .filter(s => s.name.toLowerCase().includes(search.toLowerCase()));
  }, [filter, search]);

  // Also memoize computed values
  const average = useMemo(() =>
    filteredStudents.reduce((sum, s) => sum + s.score, 0) /
    filteredStudents.length
  , [filteredStudents]);

  return (
    <div>
      <input value={search} onChange={e => setSearch(e.target.value)}
placeholder="Search..." />
      <select value={filter} onChange={e => setFilter(e.target.value)}>
        <option value="all">All</option>
        <option value="passing">Passing</option>
        <option value="failing">Failing</option>
      </select>
      <p>Showing {filteredStudents.length} students | Average:
{average?.toFixed(1)}</p>
      <ul>{filteredStudents.map(s => <li key={s.id}>{s.name}:
{s.score}</li>)}</ul>
    </div>
  );
}

```

Hook	Key Characteristic
useState	Reactive value — triggers re-render on change
useRef	Persistent value — does NOT trigger re-render
useCallback	Memoize a function — same reference across renders
useMemo	Memoize a computed value — recalculate only on dep change

🔗 Exercise 9: useRef, useReducer, useCallback, useMemo

1. Build a form that auto-focuses the first input field using useRef
2. Use useRef to store a setInterval ID and build a start/stop/reset stopwatch
3. Rebuild the shopping cart from 9.2 using useReducer with add, remove, and clear actions

4. Build a student list with search and filter. Use `useMemo` to memoize the filtered result
5. Wrap a sort function with `useCallback` and pass it to a memoized child component

UNIT 10 ADVANCED REACT — Custom Hooks · HOC · Suspense · Forward Ref · Transitions

10.1 Custom Hooks

A custom hook is a function that starts with "use" and can call other hooks. They let you extract and share stateful logic between components.

USELOCALSTORAGE — A REUSABLE CUSTOM HOOK

```
import { useState } from 'react';

// A custom hook to sync state with localStorage
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch { return initialValue; }
  });

  const setValue = (value) => {
    const valueToStore = value instanceof Function ? value(storedValue) : value;
    setStoredValue(valueToStore);
    localStorage.setItem(key, JSON.stringify(valueToStore));
  };

  return [storedValue, setValue];
}

// useFetch — custom hook for data fetching
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then(res => { if (!res.ok) throw new Error("Failed"); return res.json(); })
      .then(data => { setData(data); setLoading(false); })
      .catch(err => { setError(err.message); setLoading(false); });
  }, [url]);

  return { data, loading, error };
}

// Using custom hooks — clean and reusable
function PostList() {
  const { data: posts, loading, error } = useFetch("/api/posts");
  const [theme, setTheme] = useLocalStorage("theme", "light");
}
```

```

    if (loading) return <p>Loading...</p>;
    if (error)    return <p>Error: {error}</p>;

    return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
  }
}

```

10.2 Higher-Order Components (HOC)

A Higher-Order Component is a function that takes a component and returns a new enhanced component. It is a pattern for reusing component logic.

WITHAUTH HOC — REDIRECT IF NOT LOGGED IN

```

// A HOC that wraps any component with authentication check
function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    const { user } = useContext(AuthContext);

    if (!user) {
      return <Navigate to='/login' />;
    }

    // Pass all original props through
    return <WrappedComponent {...props} />;
  };
}

// withLoading HOC — shows spinner while loading
function withLoading(WrappedComponent) {
  return function WithLoadingComponent({ isLoading, ...props }) {
    if (isLoading) return <div className="spinner">Loading...</div>;
    return <WrappedComponent {...props} />;
  };
}

// Apply HOCs
const ProtectedPage    = withAuth(AdminPage);
const LoadingPostList = withLoading(PostList);

// Usage
<ProtectedPage />
<LoadingPostList isLoading={loading} posts={posts} />

```

10.3 React.Suspense & Lazy Loading

Suspense lets components "wait" for something — most commonly lazy-loaded components. `React.lazy()` splits your bundle and loads components only when needed.

REACT.LAZY AND SUSPENSE — CODE SPLITTING

```

import { Suspense, lazy } from 'react';

```

```
// Lazy load - the component is only downloaded when first needed
const AdminPage = lazy(() => import('./pages/AdminPage'));
const SettingsPage = lazy(() => import('./pages/SettingsPage'));

function App() {
  return (
    <BrowserRouter>
      {/* Suspense shows fallback while the lazy component loads */}
      <Suspense fallback={<div className="loading">Loading page...</div>}>
        <Routes>
          <Route path="/admin" element={<AdminPage />} />
          <Route path="/settings" element={<SettingsPage />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

10.4 Forward Ref

`forwardRef` lets a parent component access a DOM element inside a child component via a ref — needed when you build reusable input components.

FORWARDREF — CREATING REF-ABLE CUSTOM INPUTS

```
import { forwardRef } from 'react';

// A custom Input component that exposes its <input> DOM element
const CustomInput = forwardRef(function CustomInput({ label, ...props }, ref) {
  return (
    <div className="form-group">
      <label>{label}</label>
      <input ref={ref} {...props} /> {/* ref forwarded to native input */}
    </div>
  );
});

// Parent can now focus/select the inner input directly
function LoginForm() {
  const emailRef = useRef(null);

  useEffect(() => {
    emailRef.current.focus(); // focuses the <input> inside CustomInput
  }, []);

  return (
    <form>
      <CustomInput ref={emailRef} label="Email" type="email" />
    </form>
  );
}
```

10.5 useTransition — React 18

useTransition marks state updates as non-urgent, keeping the UI responsive while expensive renders happen in the background.

USETRANSITION — KEEPING THE UI RESPONSIVE

```
import { useState, useTransition } from 'react';

function SearchPage() {
  const [query,      setQuery]      = useState("");
  const [results,    setResults]    = useState([]);
  const [isPending,  startTransition] = useTransition();

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);    // urgent: update input immediately

    // Non-urgent: filtering 10,000 items can wait
    startTransition(() => {
      const filtered = bigList.filter(item =>
        item.name.toLowerCase().includes(value.toLowerCase())
      );
      setResults(filtered);
    });
  };

  return (
    <div>
      <input value={query} onChange={handleChange} placeholder="Search..." />
      {isPending && <p>Updating results...</p> }
      <ul>{results.map(r => <li key={r.id}><{r.name}</li>)}</ul>
    </div>
  );
}
```

🔗 Exercise 10: Advanced React Topics

1. Create a useFetch custom hook and use it in three different components to fetch different endpoints
2. Create a useDebounce custom hook and apply it to a search input (only search after 300ms of inactivity)
3. Build a withErrorBoundary HOC that catches errors in wrapped components and shows a friendly error message
4. Set up React.lazy and Suspense to lazy-load your AdminPage component
5. Create a reusable TextInput component with forwardRef so a parent can focus it programmatically

UNIT 11 QUICK REFERENCE — Hooks Cheat Sheet • Common Patterns • Glossary

11.1 All Hooks at a Glance

Hook Signature	What It Does
<code>useState(init)</code>	Local reactive state. Re-renders on change.
<code>useEffect(fn, deps)</code>	Side effects: fetch, timers, subscriptions.
<code>useContext(ctx)</code>	Read value from a <code>Context.Provider</code> .
<code>useRef(init)</code>	Mutable ref. Access DOM or persist values.
<code>useReducer(fn, init)</code>	Complex state with reducer/dispatch pattern.
<code>useCallback(fn, deps)</code>	Memoized function — stable across renders.
<code>useMemo(fn, deps)</code>	Memoized computed value.
<code>useTransition()</code>	Mark updates as non-urgent (React 18).
<code>useId()</code>	Generate unique IDs for accessibility.

11.2 State Update Patterns

COMMON STATE UPDATE RECIPES

```
// Counter
setCount(prev => prev + 1);

// Toggle boolean
setIsOpen(prev => !prev);

// Update one field in object
setUser(prev => ({ ...prev, name: "New Name" }));

// Add to array
setItems(prev => [...prev, newItem]);

// Remove from array by id
setItems(prev => prev.filter(item => item.id !== targetId));

// Update item in array
setItems(prev =>
  prev.map(item => item.id === targetId ? { ...item, done:true } : item)
);
```

11.3 useEffect Patterns

COMMON USEEFFECT RECIPES

```
// Fetch on mount
useEffect(() => {
  fetchData().then(setData);
}, []);

// React to prop/state change
useEffect(() => {
  doSomethingWith(userId);
}, [userId]);

// Cleanup on unmount
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);
}, []);

// async inside useEffect (must wrap)
useEffect(() => {
  async function load() {
    const res = await fetch("/api/posts");
    const data = await res.json();
    setPosts(data);
  }
  load();
}, []);
```

11.4 React Glossary

Term	Definition
Component	A function that returns JSX. The basic building block of React apps.
JSX	JavaScript XML — HTML-like syntax inside JavaScript, compiled by Babel.
Props	Data passed from parent to child. Read-only in the child.
State	Data managed inside a component. Changing state triggers re-render.
Hook	Function that adds React features (state, effects) to functional components.
Virtual DOM	React's in-memory copy of the DOM. React compares it to find changes.
Re-render	React re-runs the component function to produce updated JSX.
Controlled	Form input whose value is controlled by React state.
key	Unique identifier for list items — helps React track elements.

Fragment	<code><> </></code> — group elements without adding a DOM node.
HOC	Higher-Order Component — function that takes and returns a component.
Context	Global state mechanism that avoids prop drilling.
Lazy/Suspense	Code splitting — load components only when needed.
Memoization	Caching results of expensive calls to avoid repeated computation.

11.5 Common Mistakes & Fixes

Mistake	Fix
Mutating state directly	<code>state.count++</code> → <code>setState(p => ({...p, count: p.count+1}))</code>
Missing key in <code>.map()</code>	Add <code>key={item.id}</code> to root element inside <code>.map()</code>
Calling hooks conditionally	Hooks must be at top level — never inside if/loops
Forgetting <code>useEffect</code> deps	Missing dependency causes stale data or infinite loops
class instead of <code>className</code>	JSX uses <code>className</code> , not <code>class</code>
Calling handler on render	<code>onClick={fn()}</code> calls immediately — use <code>onClick={fn}</code>
Not preventing form default	Add <code>e.preventDefault()</code> in <code>onSubmit</code> handler

11.6 W3Schools Practice Resources

Use these W3Schools resources to reinforce what you have learned:

- React Tutorial: <https://www.w3schools.com/react/>
- React Exercises: https://www.w3schools.com/react/react_exercises.asp
- React Quiz: https://www.w3schools.com/react/react_quiz.asp
- React Syllabus: https://www.w3schools.com/react/react_syllabus.asp
- Try It Editor: <https://www.w3schools.com/react/tryit.asp>

Key W3Schools Certification

After completing all exercises and quizzes, you can earn the W3Schools "Certified React Developer" certificate at campus.w3schools.com

FINAL PROJECT: Build TheFolio Blog with React

Apply everything from this guide to build a complete blog application with these features:

- React Router with 5 pages: Home, Post Detail, New Post, Login, Register
- `useState` for form inputs, post list, and UI toggles
- `useEffect` to fetch posts from an API on mount
- `useContext` for global authentication state (login/logout)

- `useRef` for auto-focus on the post title input
- Custom `useFetch` hook for reusable API calls
- CSS Modules or Sass for component-scoped styling
- React Router's `useParams` to display individual post details
- Conditional rendering for logged-in vs guest views
- Controlled forms with validation for new post creation